

ROTEIRO EXPANDIDO — PARA ESTUDO

Ceres Diagnóstico — Defesa de Artigo PSI

Namem Rachid Jaudy Neto · IFMT Cuiabá · Junho 2026

Duração total	30 minutos (mais 10 min Q&A)
Slides	23 slides
Ritmo médio	~1 min 18 s / slide
Video	demo_ceres_1.5x.mp4 (slide 20)
Esta versao	EXPANDIDA — inclui notas de estudo (caixas amarelas) em cada slide

■ Fala ★ Enfase / ponto-chave ■ Alerta / limitacao ✓ Resultado positivo → Transicao ■ Nota de estudo

SLIDE 1 Capa

0:00–0:30

- Bom dia a todos. Meu nome é Namem Rachid, sou aluno de Engenharia da Computação aqui no IFMT Cuiabá, e hoje vou apresentar o Ceres Diagnóstico.
 - O Ceres é um sistema de diagnóstico de doenças em tomateiro que usa inteligência artificial embarcada — ou seja, a IA roda diretamente num microcontrolador pequeno e barato, sem precisar de internet.
 - O trabalho tem três frentes integradas: hardware com ESP32-S3, backend com Django e aplicativo mobile com Flutter, todos conectados via protocolo MQTT.
- *Vou começar contando o problema real que motivou tudo isso.*

■ PARA ESTUDAR / CASO PERGUNTEM:

Se perguntarem o que é TinyML: é Machine Learning em dispositivos minúsculos com poucos KB de memória. O celular tem GB — o ESP32 tem 512 KB. A IA precisa caber nesse espaço.

PSI = Projeto de Sistemas de Informação. Este artigo é parte do TCC de Engenharia da Computação no IFMT Cuiabá.

SLIDE 2 Roteiro

0:30–0:45

- Vou contar a história em oito atos: o problema, a proposta, como construímos o modelo, a grande surpresa que foi o gap entre laboratório e campo, os cinco experimentos que fizemos para tentar resolver esse gap, a decisão de arquitetura, o sistema em funcionamento, e a conclusão.
 - Pode parecer bastante coisa — mas há um fio condutor: cada etapa leva à próxima naturalmente.
- *Começando pelo problema.*

SLIDE 3 Problema e Motivação

0:45–2:00

■ O tomateiro é a terceira maior produção de vegetais do mundo — no Brasil, movimenta cerca de R\$ 10 bilhões por ano segundo a FAO 2024.

■ O problema é que doenças foliares, como requeima e septoriose, podem destruir uma lavoura inteira em poucos dias se não tratadas no início. E o diagnóstico correto depende de um agrônomo especializado.

★ **Sorriso-MT — cidade aqui do Mato Grosso — concentra uma parcela grande dos produtores de tomate do estado. A maioria são pequenos agricultores que não têm acesso regular a assistência técnica. Quando o agrônomo chega, muitas vezes a planta já está perdida.**

■ Há outro obstáculo: a conectividade rural no Brasil é instável. Uma solução que depende de internet para funcionar simplesmente não serve no campo.

■ A pergunta que fizemos foi: dá para colocar a IA dentro de um dispositivo barato, que o produtor carrega no bolso, que funciona offline e custa menos de R\$ 100?

→ *Nossa proposta é exatamente isso.*

■ **PARA ESTUDAR / CASO PERGUNTEM:**

Se perguntarem o tamanho do problema: Embrapa estima perdas de até 30-40% da produção por doenças mal diagnosticadas em pequenas propriedades.

Se perguntarem por que não usar um agrônomo via vídeo chamada: latência, custo por atendimento, cobertura de sinal no campo e disponibilidade de profissionais são barreiras reais. A IA embarcada resolve o problema na origem.

Sorriso-MT: fica a ~700 km de Cuiabá, é polo agrícola relevante. O projeto prevê validação com produtores de lá na Sprint 3.

SLIDE 4 Nossa Proposta

2:00–3:00

■ A proposta é um sistema que classifica 10 doenças do tomateiro, funciona sem internet, custa menos de R\$ 100 de hardware e tem o código completamente aberto para qualquer pesquisador reproduzir.

■ O coração do sistema é um modelo de IA de 638 Kilobytes — menos que um arquivo MP3 de 30 segundos. Esse modelo roda diretamente no microcontrolador.

■ O produtor aponta a câmera para a folha, o sistema classifica a doença em menos de 1 segundo, e exibe o resultado no aplicativo — mesmo sem sinal de celular.

→ *Para chegar nisso, precisávamos de dados. Vou mostrar como montamos o pipeline.*

■ **PARA ESTUDAR / CASO PERGUNTEM:**

638 KB de comparação: um emoji PNG tem ~5 KB. Um modelo GPT tem bilhões de parâmetros. Nosso modelo tem ~2,5 milhões de parâmetros comprimidos em INT8.

Se perguntarem por que open source: reprodutibilidade científica, outros pesquisadores podem validar, e produtores de outros estados podem adaptar para suas culturas.

SLIDE 5 Objetivos e Contribuições

3:00–4:00

■ O objetivo central é integrar três áreas — TinyML, IoT e app móvel — num único pipeline reproduzível, do campo ao diagnóstico.

■ As cinco contribuições concretas do trabalho: primeiro, pipeline completo com dez classes de doenças, código e modelos abertos. Segundo, validação real no ESP32-S3 com 692 milissegundos de

latência. Terceiro, três arquiteturas de inferência documentadas e funcionando. Quarto, validação em três datasets de campo independentes — não só no dataset de laboratório. Quinto, cinco experimentos documentados, incluindo os que não funcionaram.

★ **Esse último ponto é importante: resultado negativo documentado é contribuição científica. Evita que outros pesquisadores percam meses repetindo o mesmo erro.**

→ [Começando pelo dataset.](#)

■ **PARA ESTUDAR / CASO PERGUNTEM:**

Se perguntarem o que é IoT: Internet of Things — objetos do mundo físico conectados à internet. O ESP32 coleta dados de sensores e envia via MQTT para o servidor.

Se perguntarem o que é pipeline: sequência de etapas automatizadas. Dataset bruto → pré-processamento → treinamento → quantização → embarque → inferência → resultado para o usuário.

SLIDE 6 Dataset e Pipeline

4:00–5:00

■ O dataset que usamos é o PlantVillage, publicado por Hughes e Salathé em 2015 na PLOS ONE. É o benchmark padrão da área — 18.160 imagens de folhas de tomate em dez categorias, licença aberta CC BY 4.0.

■ Dividimos em 70% para treino, 15% para validação, 15% para teste — com uma semente aleatória fixa de 42, para que qualquer pessoa que rode o código obtenha exatamente os mesmos resultados. Isso é reprodutibilidade.

■ Aplicamos augmentation offline: rotação, espelhamento, brilho, zoom. Isso multiplicou o conjunto de treino por seis, chegando a 88.949 imagens. As 2.734 imagens de teste nunca foram usadas no treinamento — são a medida honesta do desempenho.

→ [Vamos ver como são essas dez classes.](#)

■ **PARA ESTUDAR / CASO PERGUNTEM:**

Se perguntarem o que é augmentation: aumento de dados. Pegar uma foto e criar variações dela — girar, espelhar, clarear — para o modelo aprender que a doença é a mesma independente do ângulo ou iluminação. Sem isso, o modelo vira um decoreba.

Por que seed=42? É convenção da comunidade. Garante que embaralhar aleatório gera sempre a mesma sequência. Permite que outro pesquisador reproduza exatamente o mesmo split.

PlantVillage: imagens tiradas em fundo cinza controlado em laboratório. Isso vai ser o problema mais tarde (gap lab-campo).

SLIDE 7 As 10 Classes

5:00–5:30

■ Estas são as dez classes reais que o modelo reconhece — cada imagem é do PlantVillage.

■ Veja a diversidade visual: Requeima tem lesões úmidas escuras com bordas irregulares. Septoriose tem pontinhos brancos com halo amarelo — parece uma pintura pontilhista. Pinta-preta tem manchas concêntricas como anéis de árvore. Vira-cabeça deixa a folha toda amarela e enrolada. A folha saudável é verde uniforme.

■ Essa diversidade visual é o que torna o problema difícil — e interessante.

→ [Antes de treinar, precisei escolher qual arquitetura de rede usar. Próximo slide.](#)

■ PARA ESTUDAR / CASO PERGUNTEM:

Nomes técnicos das classes: D01=requeima (Phytophthora infestans), D02=septoriose (Septoria lycopersici), D03=pinta-preta (Alternaria solani), D03b=mancha-alvo (Corynespora cassiicola), D05=mofo foliar (Passalora fulva), D06=vira-cabeça (TSWV), D06b=mosaico (ToMV), D07=ácaro (Tetranychus urticae), D09=mancha bacteriana (Xanthomonas campestris), saudável.

Se perguntarem sobre doenças não incluídas: focamos nas 10 mais prevalentes no PlantVillage. Murcha bacteriana e nutricionais ficaram fora por falta de dados suficientes.

SLIDE 8 Comparativo de Modelos

5:30–6:30

■ Antes de treinar, precisei responder: qual rede neural cabe num microcontrolador de R\$ 80?

■ Avaliamos cinco arquiteturas. ResNet-50 e EfficientNet-B0 são ótimas para servidores, mas grandes demais para o ESP32-S3 — simplesmente não há suporte na biblioteca TFLite Micro. YOLO foi descartado de imediato: ele é um detector de objetos, identifica onde está o objeto na imagem com uma caixa. Nossa tarefa é diferente — classificar a folha inteira. Além disso, o menor YOLO tem 6 MB, dez vezes maior que nosso modelo.

★ **MobileNetV2 foi projetado especificamente para dispositivos móveis e embarcados. Usa uma técnica chamada 'depthwise separable convolution' que processa a imagem de forma muito mais eficiente. Resultado: mesma acurácia, fração do custo computacional. É a única opção com suporte comprovado no ESP32-S3, arena de apenas 200 KB dos 512 KB disponíveis.**

→ *Com a arquitetura escolhida, vamos ao treinamento.*

■ PARA ESTUDAR / CASO PERGUNTEM:

Depthwise separable convolution em termos simples: em vez de aplicar um filtro na imagem inteira de uma vez (caro), aplica um filtro por canal de cor separado (barato) e depois combina. Reduz operações em ~8-9 vezes sem perder muita qualidade.

Arena TFLite Micro: é a memória RAM que o modelo usa durante a inferência. O MobileNetV2 INT8 usa 200 KB dos 512 KB disponíveis no ESP32-S3 — 39%. Sobram ~290 KB livres para WiFi, MQTT e sistema operacional.

Se perguntarem por que não EfficientNet: na época dos experimentos, o suporte de EfficientNet no TFLite Micro para INT8 era instável e exigia arena maior que 512 KB.

SLIDE 9 Treinamento e Quantização

6:30–9:00

■ O treinamento aconteceu em duas fases — isso se chama transfer learning, ou aprendizado por transferência.

■ Na Fase 1, usamos o MobileNetV2 já treinado no ImageNet — 1,2 milhão de imagens de objetos do cotidiano. Congelamos esse conhecimento base e treinamos só as últimas camadas, que são responsáveis por classificar as doenças específicas. Dez épocas, taxa de aprendizado 0,001.

■ Na Fase 2, descongelamos as últimas 30 camadas e deixamos o modelo afinar todo o conhecimento com taxa de aprendizado menor — 0,0005 — por quarenta épocas. O melhor resultado foi 97,90% de acurácia na época 46.

■ Agora vem a parte crítica: quantização. O modelo treinado está em ponto flutuante de 32 bits — FP32. Para caber no ESP32 e rodar rápido, precisamos converter para inteiros de 8 bits — INT8. Isso comprime o modelo de 2,5 MB para 638 KB e acelera a inferência em 3x.

■ O problema: se você fizer a conversão de forma automática, sem calibração, o modelo perde muito desempenho. O Edge Impulse fez isso — resultado: 62% de acurácia, queda de 30 pontos percentuais. Isso é o Exp A.

✓ A solução foi calibrar a quantização com 50 batches do nosso próprio dataset de validação, chamados de 'representative dataset'. O TensorFlow usa essas amostras para entender a faixa de valores que o modelo processa e faz a conversão muito mais precisa. Resultado: queda de apenas 2,37 pontos. O modelo INT8 entrega 95,76% — quase igual ao FP32.

→ Excelente no laboratório. O problema veio quando testamos em campo.

■ PARA ESTUDAR / CASO PERGUNTEM:

Transfer learning em termos simples: como aprender a dirigir um caminhão depois de já saber dirigir carro. Você não começa do zero — aproveita o que já sabe e adapta.

INT8 vs FP32: FP32 = número com casas decimais (ex: 0.7423...). INT8 = número inteiro de -128 a 127. A conversão é uma compressão — perde um pouco de precisão. Com calibração, a perda é mínima.

Se perguntarem por que 50 batches: é o suficiente para cobrir a distribuição estatística dos dados sem demorar. Com menos, a calibração é ruim. Com mais, não melhora significativamente.

EarlyStopping: o treinamento para automaticamente se a acurácia de validação não melhorar por 10 épocas seguidas. ReduceLROnPlateau: reduz a taxa de aprendizado pela metade se estagnar. Evitam overfitting (modelo que decora o treino mas não generaliza).

SLIDE 10

Matriz de Confusão

9:00–10:00

■ Esta é a matriz de confusão do modelo INT8 no test set. Como ler: cada linha é a classe real, cada coluna é o que o modelo previu. A diagonal principal — os quadrados mais escuros — são os acertos.

■ O resultado geral: 2.618 acertos de 2.734 imagens. 95,76%. As classes com quase 100% de acerto são mancha bacteriana, vira-cabeça e saudável — padrões visuais muito distintos, fáceis de separar.

■ A classe mais difícil é pinta-preta, com cerca de 83% de acerto. Ela se confunde principalmente com septorrose e requeima — as três têm lesões escuras de borda irregular. É biologicamente parecida.

■ Esse padrão de confusão entre lesões necróticas — pinta-preta, septorrose, mancha bacteriana — persistiu em todos os cinco experimentos. É uma limitação real do modelo. Para o produtor, confundir essas três ainda é útil: o tratamento é parecido.

→ Agora — o resultado em campo real foi uma surpresa.

■ PARA ESTUDAR / CASO PERGUNTEM:

Como ler matriz de confusão: se a linha de 'pinta-preta' tem valores altos nas colunas de 'septorrose' e 'requeima', significa que o modelo confundiu pinta-preta com essas doenças.

Por que a confusão entre doenças necróticas não é catastrófica agronomicamente: septorrose, pinta-preta e mancha bacteriana têm protocolos de tratamento com fungicidas de amplo espectro similares. O erro de classificação entre elas não leva a tratamento errado necessariamente.

Se perguntarem sobre precisão vs recall: precisão é 'das vezes que o modelo disse X, quantas eram X?'. Recall é 'das imagens de X, quantas o modelo acertou?'. A matriz de confusão permite calcular ambos por classe.

SLIDE 11

Gap Laboratório–Campo

10:00–12:00

■ Até aqui, tudo ótimo: 95,76% no teste de laboratório. Aí fizemos o que qualquer pesquisa séria deve fazer — testamos em dados de campo real.

■ Usamos o PlantDoc — 1.353 imagens capturadas em condições reais: fotografos diferentes, iluminação solar variável, folhas com galhos ao fundo, ângulos tortos, câmeras de celular com qualidade variada.

★ **95,76% em laboratório. 20,77% em campo. Uma queda de 75 pontos percentuais. Isso é devastador.**

■ Mas esse não é um problema exclusivo nosso. Mohanty 2016 publicou 99% no PlantVillage — o mesmo artigo seminal que popularizou o uso de CNNs para doenças de plantas. Singh 2020 testou o mesmo tipo de modelo no PlantDoc e obteve cerca de 31%. Xu 2024 revisou 42 trabalhos diferentes e documentou quedas entre 29 e 58 pontos ao ir para campo.

■ *O fenômeno tem nome: domain shift, ou mudança de domínio. O modelo aprendeu features erradas — aprendeu que 'fundo cinza = laboratório = folha de tomate saudável ou doente'. No campo, o fundo é verde, terra, galhos. O modelo fica perdido.*

■ A boa notícia: identificamos a causa. Agora era investigar como resolver.

→ *Então abrimos cinco experimentos sistemáticos.*

■ PARA ESTUDAR / CASO PERGUNTEM:

Domain shift em termos simples: imagine aprender a reconhecer gatos só vendo fotos de gatos brancos em fundo preto. Na hora que aparecer um gato laranja num sofá, você falha — não porque gatos laranjas sejam difíceis, mas porque seu treinamento foi limitado. O modelo fez o mesmo com as doenças.

Por que o PlantVillage usa fundo cinza? Controle científico — elimina variáveis externas para focar só na folha. Mas isso criou um atalho indesejado para o modelo.

Se perguntarem por que 20,77% é pior que aleatório: são 10 classes, então aleatório seria 10%. 20,77% é melhor que aleatório, mas inaceitável para uso real. O modelo acerta alguma coisa, mas erra demais para confiar.

SLIDE 12

Os 5 Experimentos

12:00–14:30

■ Fizemos cinco experimentos para entender e atacar o gap. Vou contar cada um como uma história — o que tentamos, o que esperávamos e o que aconteceu.

■ Experimento A: Edge Impulse, sem calibração INT8. Resultado: 62% em laboratório. Esse experimento confirmou o problema da quantização automática e serviu como linha de base do que NÃO fazer. Não chegamos nem a testar em campo.

■ Experimento B: TensorFlow local com calibração INT8. 95,76% em laboratório, 20,77% em campo. Esse é o modelo base — a referência a partir da qual todos os outros se comparam.

■ *Experimento C: augmentation sintética de fundo. A ideia era: se o problema é o fundo cinza, vamos trocar o fundo de todas as imagens por fundos naturais. Geramos 177.698 composições. Resultado: 20,24% em campo — pior do que o Exp B. Resultado negativo. Vou detalhar por quê.*

■ Experimento D: fine-tuning com dados reais de campo. Pegamos 677 imagens reais do PlantDoc e retunamos o modelo. Resultado: 30,43% em campo — +10 pontos. Primeira melhora real.

✓ Experimento E: Focal Loss com $\gamma=2$ somado ao Exp D. Resultado: 27,65% no Tomato-Village indiano — +16 pontos sobre o baseline. É o modelo final embarcado.

→ Vou detalhar o Exp C — o resultado negativo é o mais pedagógico.

■ PARA ESTUDAR / CASO PERGUNTEM:

Por que documentar experimentos negativos? Porque 80% da ciência que deu errado não é publicada. Isso cria um viés: pesquisadores do mundo inteiro repetem os mesmos erros. Publicar o fracasso do Exp C é contribuição genuína.

Linha do tempo dos experimentos: A e B foram os primeiros (durante Sprint 1). C foi feito após identificar o gap. D e E foram iterações sobre o que funcionou em D.

SLIDE 13

Exp C — Resultado Negativo

14:30–16:00

■ O Experimento C foi nossa aposta mais ambiciosa — e a que mais falhou.

■ A lógica era clara: se o modelo aprende o fundo cinza como feature, substituir o fundo por imagens naturais deveria forçá-lo a focar nas lesões da folha. Usamos a biblioteca rembg, que usa uma rede neural chamada U2-Net para separar o objeto do fundo automaticamente. Geramos 177.698 imagens compostas — folhas do PlantVillage sobre fundos naturais do PlantDoc.

■ Resultado: 20,24% no PlantDoc — 0,5 pontos PIOR que o Exp B. E a classe 'saudável' caiu para 0% de acerto em campo. Zero.

■ O que aconteceu? Dois problemas. Primeiro: a rembg não é perfeita — ela deixa artefatos de borda nas folhas, uma espécie de halo artificial. O modelo aprende esse artefato como feature. Segundo: a iluminação da folha e do fundo novo são inconsistentes — o fundo foi tirado com luz solar, a folha com luz de laboratório. O modelo percebe a composição como falsa.

★ A lição foi clara: síntese não substitui dado real. O modelo sabe distinguir imagem sintética de imagem real. Só o fine-tuning com imagens genuínas de campo foi efetivo.

→ Vamos ver o que funcionou — Exp D e E.

■ PARA ESTUDAR / CASO PERGUNTEM:

U2-Net é uma rede de segmentação — ela delimita os contornos de objetos numa imagem. É boa, mas imperfeita em folhas com bordas irregulares e transparência parcial.

Por que a classe saudável zerou? O modelo treinado com fundos sintéticos passou a associar 'folha sobre fundo natural = doença' — porque saudável no PlantVillage tem fundo cinza impecável, e a composição sintética ficou 'parecida com doença' para o modelo.

Se perguntarem se tentamos outros geradores de fundo: a literatura (Singh 2020) documenta resultado similar com GAN para augmentation de fundo. A composição sintética é uma limitação geral do campo, não específica da nossa implementação.

SLIDE 14

Exp D e E — O Que Funcionou

16:00–17:30

■ O Experimento D foi simples e direto: pegar imagens reais de campo e usá-las no treinamento.

■ Usamos 677 imagens do conjunto de treino do PlantDoc, repetidas 10 vezes para equilibrar com o volume do PlantVillage. As 69 imagens de teste do PlantDoc nunca foram vistas. Resultado: 30,43% — +10 pontos sobre o Exp B. Primeira melhora concreta.

■ A conclusão do Exp D: o fator limitante não é a técnica de treinamento — é o volume de dados de campo. Com 677 imagens reais, já ganhamos 10 pontos. Com 10.000 imagens reais brasileiras, o salto seria muito maior.

✓ O Experimento E adicionou Focal Loss com $\gamma=2$. O que é isso? É uma função de perda que pune mais os erros nas imagens difíceis e reduz o peso das imagens fáceis. As imagens de laboratório com fundo cinza são 'fáceis' para o modelo — ele as acerta de olhos fechados. Com Focal Loss, elas contribuem menos para o aprendizado, forçando o modelo a se concentrar nas imagens de campo, que são mais difíceis. Resultado no Tomato-Village: 27,65% — +16 pontos sobre o baseline.

→ Mas esse ganho não se distribuiu igualmente pelo mundo.

■ PARA ESTUDAR / CASO PERGUNTEM:

Focal Loss foi introduzida por Lin et al. 2017 no paper RetinaNet para detecção de objetos. Nós a adaptamos para classificação de doenças. O $\gamma=2$ é o valor padrão recomendado pelos autores.

Por que repetir as 677 imagens 10 vezes? Para balancear com o volume do PlantVillage. Sem isso, as imagens de campo seriam uma gota no oceano e o modelo as ignoraria.

Se perguntarem por que não juntar D e E e testar no PlantDoc: o Exp E foi avaliado principalmente no Tomato-Village, que tem características diferentes. No PlantDoc, o ganho do Exp E sobre o D foi menor — mas consistente.

SLIDE 15

Gap Geográfico e Colapso de Classe

17:30–18:30

■ Um achado que não esperávamos: a acurácia em campo não é uniforme — ela cai conforme a distância geográfica do dataset de treinamento.

■ PlantDoc, majoritariamente EUA e Europa: 30,43%. Tomato-Village, Índia: 27,65%. Daffodil, Bangladesh: 18,13%. Quanto mais longe geográfica e climaticamente do PlantVillage americano, pior o modelo performa.

■ Barbedo 2019 previu exatamente isso em seu survey: variedades locais de tomate, iluminação tropical diferente, estágio fenológico da planta, e até o solo da região afetam como a doença se manifesta visualmente.

■ O caso mais extremo: no Tomato-Village, 73% das folhas saudáveis indianas foram classificadas como septoriose. Setenta e três por cento. O modelo criou um atalho errado — 'folha com textura indiana = septoriose'. Isso se chama colapso de classe. O Exp E com Focal Loss mitigou esse problema parcialmente.

→ Com o modelo definido, a questão passou a ser: onde esse modelo vai rodar?

■ PARA ESTUDAR / CASO PERGUNTEM:

Por que a doença parece diferente em climas diferentes? A mesma bactéria ou fungo se manifesta de formas distintas dependendo de temperatura, umidade, variedade da planta e estágio de infecção. Uma requeima no MT em novembro parece diferente de uma requeima em Ohio em setembro.

Colapso de classe: quando o modelo 'desiste' de tentar classificar certas classes e joga tudo numa categoria. Acontece quando o domain shift é muito extremo — o modelo não sabe como lidar com o dado e chuta o mais comum.

Implicação direta: um dataset brasileiro de campo é essencial. Imagens de Sorriso-MT, coletadas nas condições reais do MT, teriam impacto muito maior que qualquer técnica de augmentation.

SLIDE 16

Modelo Pronto — Onde Rodar?

18:30–20:30

■ Com o modelo treinado e quantizado, a pergunta de engenharia foi: onde esse modelo vai rodar na prática?

■ Identificamos três caminhos, todos usando o mesmo arquivo de 638 KB. Caminho 1: diretamente no ESP32-S3, sem nenhum outro dispositivo. Caminho 2: no smartphone Android do produtor, via `tflite_flutter`. Caminho 3: na nuvem, via API Django hospedada no Railway.

■ Cada caminho tem um perfil diferente. Edge é o mais autônomo e privado — a imagem nunca sai do dispositivo. Mobile é o mais acessível — qualquer produtor já tem um celular. Cloud é o mais potente — servidor com mais memória, mais rápido, mas requer internet.

★ **Aqui precisamos contar um pivô de engenharia importante. O plano original era o ESP32-S3 com a câmera OV5640, completamente autônomo — o produtor aponta a câmera acoplada ao microcontrolador para a folha e recebe o diagnóstico. Durante a integração, percebemos que a câmera OV5640 precisa de calibração específica de buffer de captura e correção de exposição — trabalho de Sprint 2. A adaptação foi usar o smartphone como câmera no Caminho 2, enquanto o ESP32 assume o papel de sensor de ambiente.**

→ *Vamos comparar os três caminhos lado a lado.*

■ PARA ESTUDAR / CASO PERGUNTEM:

OV5640: câmera de 5MP compatível com ESP32-S3. O problema não é a câmera em si — é o pipeline de captura: bufferizar os frames JPEG, corrigir exposição em tempo real, e passar para o modelo sem perder latência. É resolvível, mas exige mais um sprint.

tflite_flutter 0.12.1: biblioteca que roda modelos TFLite diretamente no Android, sem servidor. O mesmo arquivo .tflite do ESP32 funciona no celular sem modificação alguma — essa é a elegância da padronização TFLite.

Se perguntarem por que não usar só cloud: conectividade rural, latência variável, custo de servidor, e privacidade dos dados do produtor. O edge e o mobile resolvem os três problemas.

SLIDE 17

Comparativo Edge / Mobile / Cloud

20:30–21:30

■ Comparando os três caminhos: Edge no ESP32-S3 entrega 692 milissegundos determinísticos — sempre o mesmo tempo, com variação de apenas 1 ms. Medimos em 10 inferências consecutivas, todas corretas. É 2 vezes mais rápido que a estimativa do Edge Impulse.

■ Mobile estimamos entre 200 e 400 ms — não medimos empiricamente neste ciclo, é uma limitação documentada. Cloud Django mediu 306 ms via HTTPS no Railway.

■ Em termos de custo de infraestrutura: ESP32-S3 custa cerca de R\$ 80. Mobile é zero — usa o celular que o produtor já tem. Cloud usa o free tier do Railway — também zero para protótipo.

★ **O ponto central: o mesmo arquivo de 638 KB roda nos três ambientes sem modificação. Isso é o poder da padronização TFLite.**

→ *Agora deixa eu mostrar o que de fato construímos.*

■ PARA ESTUDAR / CASO PERGUNTEM:

692 ms: o produtor leva de 2 a 5 segundos para posicionar a folha corretamente. 692 ms é imperceptível na prática. A meta de 300 ms é para futura otimização com o pipeline de câmera completo — não é um bloqueador atual.

Por que a latência mobile não foi medida: exigiria um dispositivo Android físico com o APK instalado e instrumentação de profiling. Fica para Sprint 3 com validação de campo.

Railway free tier: 500 horas/mês de execução, banco PostgreSQL com 1 GB. Suficiente para protótipo e demonstração. Para produção com múltiplos produtores exigiria upgrade.

SLIDE 18

O Que Construímos

21:30–22:30

- Este slide conta a arquitetura completa do que foi construído — quatro camadas integradas.
 - Camada 1 — Firmware: ESP32-S3 com PlatformIO e ESP-IDF 5.x. O ESP32 lê temperatura e umidade do ar com o sensor DHT22, e umidade do solo com sensor capacitivo. Publica esses dados via MQTT com QoS 1 — garantia de entrega — e TLS 1.3 — criptografia.
 - Camada 2 — Broker: HiveMQ Cloud na porta 8883 com TLS. O broker é o intermediário que recebe as mensagens do ESP32 e distribui para quem estiver ouvindo — no caso, o backend Django.
 - Camada 3 — Backend: Django REST com 12 endpoints, autenticação JWT, listener MQTT que persiste cada evento com timestamp e GPS no PostgreSQL hospedado no Railway.
 - Camada 4 — App: Flutter com mais de dez telas, banco local Drift SQLite para funcionar offline, tflite_flutter 0.12.1 para inferência local, sincronização automática quando reconecta.
- ★ O arquivo `ceres_mobilenetv2_int8.tflite` — 638 KB — é o mesmo no ESP32-S3 e no Android. Um único modelo, dois ambientes completamente diferentes.

→ [Vamos ver esse sistema funcionando.](#)

■ PARA ESTUDAR / CASO PERGUNTEM:

QoS 1 no MQTT: Quality of Service nível 1. Garante que a mensagem chegue ao broker pelo menos uma vez. QoS 0 é 'fire and forget' — pode perder. QoS 2 é 'exatamente uma vez' — mais lento. QoS 1 é o equilíbrio certo para sensores de campo.

JWT = JSON Web Token. É o padrão para autenticação sem sessão: o app recebe um token criptografado ao fazer login, e manda esse token em cada requisição. O servidor valida o token sem precisar consultar o banco. Stateless, escalável.

Drift SQLite: biblioteca Flutter para banco de dados local. Permite que o app salve diagnósticos e dados de sensor offline e sincronize com o servidor quando a internet voltar. Resolve o problema de conectividade rural.

Se perguntarem por que HiveMQ e não Mosquitto local: HiveMQ Cloud tem infraestrutura gerenciada, TLS configurado, e free tier para até 100 conexões. Para protótipo com produtores dispersos geograficamente, é a solução certa.

SLIDE 19

O Sistema em Funcionamento

22:30–23:30

- À esquerda, a tela IoT do app Flutter. Mostra em tempo real: 29,5 graus Celsius, 49% de umidade do ar, 34% de umidade do solo, e status ONLINE — o ESP32 está publicando dados.
- Abaixo, o hardware físico: o ESP32-S3 conectado ao DHT22 para temperatura e umidade do ar, e ao sensor capacitivo enterrado no solo para umidade.

■ O pipeline completo em funcionamento: ESP32 lê os sensores a cada 30 segundos, publica via MQTT para o HiveMQ Cloud, o listener Django recebe, persiste no PostgreSQL com timestamp e coordenada GPS do dispositivo, o app Flutter sincroniza e exibe — tudo em menos de 2 segundos de ponta a ponta.

→ [Vou mostrar o vídeo demonstrativo agora.](#)

■ PARA ESTUDAR / CASO PERGUNTEM:

Sensor DHT22 vs DHT11: DHT22 tem maior precisão ($\pm 0,5^{\circ}\text{C}$ e $\pm 2\%$ umidade) e faixa de operação maior. Custo marginal de ~R\$ 5 a mais. Para aplicação agrícola de precisão, vale.

Sensor de solo capacitivo vs resistivo: o capacitivo não tem contato metálico com o solo, então não corrói. O resistivo corrói em 6-12 meses em solo úmido. Para uso em campo permanente, capacitivo é obrigatório.

Se perguntarem sobre GPS: a localização é obtida pelo app Flutter via Location API do Android e enviada ao Django junto com cada diagnóstico. O ESP32 não tem GPS — não seria custo-benefício para sensores de ambiente.

SLIDE
20

Demonstração

23:30–25:30

■ O vídeo que vou mostrar está em velocidade 1,5 vezes — dura cerca de 1 minuto. Mostra quatro coisas: captura de uma folha pela câmera do app, diagnóstico pelos três caminhos de inferência, publicação de dados pelo ESP32 via MQTT, e o histórico de diagnósticos com localização no mapa.

■ *Se o vídeo não abrir: o arquivo `demo_ceres_1.5x.mp4` está na pasta `docs/slides`. Abrir diretamente no player.*

→ [Após o vídeo, conclusão.](#)

■ PARA ESTUDAR / CASO PERGUNTEM:

Durante o vídeo, observe: (1) o tempo de resposta da inferência local no app; (2) a tela de histórico com mapa mostrando os pontos de diagnóstico; (3) a tela IoT atualizando os valores de sensor em tempo real.

Se a banca pedir para demonstrar ao vivo: o APK está no GitHub, o backend está no Railway. Precisaria de celular Android e ESP32 conectado na mesma rede WiFi. Improvável na defesa, mas possível.

SLIDE
21

Conclusão

25:30–27:30

★ **A manchete deste trabalho: TinyML agrícola funciona — o gargalo é o dado, não o hardware.**

■ Card 1 — Viabilidade provada: conseguimos rodar um classificador de 10 doenças em 638 KB e 692 ms no ESP32-S3 de R\$ 80. A calibração INT8 foi a técnica que viabilizou isso — sem ela, perderíamos 30 pontos de acurácia. Pipeline completo integrado e funcionando.

■ Card 2 — O gap lab-campo é um fenômeno estrutural da área: 95,76% no laboratório caindo para 18-30% no campo não é fracasso deste trabalho — é a realidade documentada por Xu 2024 em 42 trabalhos. Identificamos a causa e o caminho: fine-tuning com dados reais funciona. O problema é que dados reais brasileiros simplesmente não existem em volume suficiente.

■ *Card 3 — O gargalo real não é o microcontrolador, não é a rede neural, não é o protocolo MQTT. É a ausência de um dataset brasileiro de campo com variedades locais, iluminação tropical e condições reais de Mato Grosso. Nenhuma técnica de síntese substitui esse dado.*

→ [O que vem a seguir.](#)

■ PARA ESTUDAR / CASO PERGUNTEM:

Se a banca desafiar os 18-30% como insuficiente: concordar que é insuficiente para uso clínico, mas contextualizar: (1) é um primeiro ciclo; (2) a literatura mostra o mesmo padrão; (3) o caminho para resolver está identificado; (4) 30% já é 3 vezes melhor que aleatório e já dá dica útil para o produtor sobre o que investigar.

Se perguntarem se vale lançar com 30% de acurácia: sim, como ferramenta de triagem — 'provável doença fúngica, chamar agrônomo' — não como diagnóstico definitivo. A IA apoia o produtor, não substitui o especialista.

Focal Loss: Lin et al. 2017, ICCV. Citação completa no artigo.

SLIDE 22

Trabalhos Futuros

27:30–29:00

■ Quatro próximos passos claros.

■ Primeiro e mais urgente: coletar um dataset brasileiro de campo com produtores de Sorriso-MT. Imagens de folhas doentes nas condições reais do Mato Grosso — iluminação tropical, variedades locais, diferentes estágios de infecção. Esse dataset é o fator que mais limita a generalização do modelo. Com 5.000 imagens reais do MT, a acurácia em campo deve superar 60%.

■ Segundo: integrar a câmera OV5640 ao ESP32-S3, completando o ciclo embarcado autônomo. O firmware de captura e o pipeline de inferência on-device estão prontos — falta calibrar o buffer da câmera. Sprint 2.

■ Terceiro: validar com produtores reais. Levar o sistema para propriedades em Sorriso-MT, medir usabilidade, acurácia em condições de campo real, e colher feedback sobre o fluxo de uso. Sprint 3.

■ Quarto: expandir para outras culturas. O pipeline é genérico — soja, milho e café são candidatos naturais dada a importância econômica no MT.

→ [Para encerrar.](#)

■ PARA ESTUDAR / CASO PERGUNTEM:

Por que 5.000 imagens seria suficiente? Com fine-tuning sobre o Exp E, 677 imagens deram +10 pp. Projeção linear conservadora: 5.000 imagens dariam +50-70 pp — levando a acurácia de campo para 70-85%. Não linear na prática, mas a ordem de grandeza é essa.

Custo da coleta: câmera de celular + protocolo de coleta padronizado (ângulo fixo, distância fixa, três fotos por folha). Um dia no campo com produtor colaborador geraria 300-500 imagens. 10 dias de campo = dataset suficiente.

SLIDE 23

Obrigado / Perguntas

29:00–30:00

■ O Ceres Diagnóstico demonstra que TinyML embarcado é viável para diagnóstico agrícola de baixo custo. O principal desafio aberto não é tecnológico — é de dados. O caminho está identificado.

■ Todo o código, os modelos treinados, os scripts de avaliação e a documentação estão no GitHub: github.com/Namem/extensao2. Qualquer pesquisador pode reproduzir ou estender este trabalho.

★ Fico à disposição para perguntas.

■ PARA ESTUDAR / CASO PERGUNTEM:

Respirar fundo antes das perguntas. Se não souber responder algo: 'Boa pergunta. Não tenho os dados para afirmar com certeza, mas a minha hipótese é...' É melhor que inventar.

Perguntas esperadas mais prováveis: (1) Por que não YOLO? (2) O gap de 75 pp não invalida a proposta? (3) A câmera não integrada não é uma limitação grave? (4) Como garantir privacidade dos dados? — Todas têm resposta boa no FAQ abaixo.

PERGUNTAS FREQUENTES — Q&A;

Q: Por que não usar YOLO?

R: YOLO é um detector de objetos — ele retorna uma caixa delimitadora (bounding box) ao redor do objeto, não uma classe única. Nossa tarefa é classificar a folha inteira: 'qual doença está nesta folha?' — não 'onde está a lesão?'. São problemas diferentes. Além disso, o menor YOLO tem ~6 MB versus nossos 638 KB — 10x maior. Não cabe no ESP32-S3 e não resolve nosso problema.

Q: Por que MobileNetV2 e não ResNet, EfficientNet ou Vision Transformer?

R: MobileNetV2 foi projetado especificamente para dispositivos com restrição de memória e processamento — depthwise separable convolutions reduzem FLOPs em 8-9x com mínima perda de qualidade. ResNet-50 tem ~25 MB de parâmetros — não há suporte TFLite Micro. EfficientNet-Lite exige arena maior que 512 KB. Vision Transformers são ainda maiores. Vision em borda em 2024 ainda é dominado por MobileNets — e a literatura de doenças de plantas usa majoritariamente MobileNetV2 e similares exatamente por esse motivo.

Q: Gap de 75 pontos não invalida a proposta do sistema?

R: Não, por três razões. Primeira: o fenômeno é documentado na literatura — Xu 2024 revisou 42 trabalhos e encontrou quedas de 29-58 pp ao ir para campo. Não é falha deste trabalho, é o estado-da-arte do campo. Segunda: o caminho para resolver foi identificado — fine-tuning com dados reais funciona (Exp D: +10 pp, Exp E: +16 pp). O problema é volume de dados de campo, não o método. Terceira: mesmo com 20-30%, o sistema é útil como ferramenta de triagem — indica 'provável doença fungica, acionar agrônomo' com muito mais informação que nada. A IA apoia, não substitui o especialista.

Q: A câmera OV5640 não integrada não é uma limitação crítica?

R: É uma limitação honestamente documentada — não escondida. A decisão foi isolar a latência pura da CNN (692 ms) carregando imagens como array em C, sem o overhead da câmera. A integração da câmera (calibração de buffer, correção de exposição, latência de leitura) é trabalho de Sprint 2. O smartphone serve como sensor alternativo no caminho móvel com o mesmo modelo — de fato aumentando a acessibilidade, pois o produtor já tem o celular.

Q: 692 ms é lento para uso em campo?

R: Não. O produtor leva de 2 a 5 segundos para posicionar a folha adequadamente. 692 ms é imperceptível na prática. A meta futura de 300 ms é para quando o pipeline de câmera estiver completo e for necessário inferência contínua (video stream). Para o caso de uso atual — foto única de folha — 692 ms é mais que suficiente.

Q: Como funciona a privacidade dos dados do produtor?

R: Nos caminhos Edge (ESP32-S3) e Mobile (Android), a imagem nunca sai do dispositivo — apenas o resultado textual da classificação é transmitido. No caminho Cloud, a imagem trafega via HTTPS com TLS 1.3 para o servidor Django. O produtor pode escolher o caminho. Para conformidade máxima com LGPD, os caminhos Edge e Mobile são os adequados — a imagem da lavoura nunca vai para um servidor externo.

Q: O sistema funciona sem internet?

R: Caminhos Edge e Mobile: 100% offline. O app Flutter armazena diagnosticos e dados de sensor no banco local Drift SQLite. Quando o usuario reconecta, a sincronizacao com o servidor acontece automaticamente em background. Caminho Cloud: requer conectividade. Para o cenario de producao com pequenos agricultores rurais, o foco e nos caminhos offline.

Q: Por que calibrar INT8 com representative_dataset?

R: Na quantizacao INT8, os pesos do modelo (numeros FP32) sao mapeados para inteiros de -128 a 127. Esse mapeamento precisa conhecer a faixa de valores que o modelo processa na pratica — caso contrario, valores raros fora da faixa calibrada sao cortados (clipping) e a acuracia despenca. O representative_dataset sao 50 batches do validation set que mostram ao conversor TFLite a distribuicao real dos valores. Com calibracao: queda de 2,37 pp. Sem calibracao (Edge Impulse automatico): queda de 30,5 pp.

Q: O que e Focal Loss e por que funciona para o gap lab-campo?

R: Focal Loss (Lin et al. 2017) e uma funcao de perda que reduz o peso dos exemplos faceis durante o treinamento. As imagens de laboratorio com fundo cinza sao 'faceis' — o modelo as acerta com alta confianca. Na loss padrao (cross-entropy), elas dominam o gradiente e o modelo se especializa nelas. Com Focal Loss ($\gamma=2$), o peso dessas imagens faceis cai para quasi zero, e o modelo foca nas imagens dificeis — que sao exatamente as de campo. Resultado: +16 pp no Tomato-Village.

Q: Por que o Exp C (augmentation sintetica) falhou?

R: Dois motivos principais. Primeiro: a rembg (U2-Net) deixa artefatos de borda nas folhas — halos artificiais que o modelo aprende como feature. Segundo: a iluminacao da folha (luz de laboratorio) e do novo fundo (foto de campo) sao inconsistentes — o modelo percebe a composicao como falsa e aprende um novo atalho errado. A literatura documenta resultado similar: Singh 2020 tambem nao conseguiu superar o fine-tuning com dados reais usando augmentation sintetica de fundo.

Q: O que e transfer learning e por que foi usado?

R: Transfer learning e usar um modelo ja treinado numa tarefa grande (ImageNet — 1,2 milhao de imagens de 1.000 categorias) como ponto de partida para uma tarefa menor e especifica (10 doencas de tomate). As camadas iniciais do MobileNetV2 ja aprenderam a detectar bordas, texturas e padroes visuais basicos — validos para qualquer imagem. Treinar essas camadas do zero exigiria muito mais dados e tempo. Com transfer learning, so retreinamos as camadas finais de classificacao (Fase 1) e depois refinamos todo o modelo (Fase 2).

Q: Como sera feita a validacao com produtores reais?

R: Sprint 3 prevê idas a campo em Sorriso-MT com produtores voluntarios. O protocolo: instalar o app no celular do produtor, acompanhar o uso durante uma semana, coletar imagens rotuladas por agroonomo parceiro, e avaliar usabilidade (SUS — System Usability Scale) e acuracia em condicoes reais. Esse dataset brasileiro coletado sera o principal insumo para o Exp F — proximo ciclo de treinamento.
